

1. Because `purse` contains a `WizCoin` object, the `WizCoin` class's `__mul__()` method is called with `2` passed for the `other` parameter.
2. The `__mul__()` method creates a new `WizCoin` object and returns it.
3. This returned object is what the `purse * 2` expression evaluates to.

Numeric dunder methods and reflected numeric dunder methods have identical code if they are *commutative*. Commutative operations, like addition, have the same result backward and forward: $3 + 2$ is the same as $2 + 3$. But other operations aren't commutative: $3 - 2$ is not the same as $2 - 3$. Any commutative operation can just call the original numeric dunder method whenever the reflected numeric dunder method is called. For example, add the following to the end of the `wizcoin.py` file to define a reflected numeric dunder method for the multiplication operation:

```
--snip--
def __rmul__(self, other):
    """Multiplies the coin amounts by a non-negative integer."""
    return self.__mul__(other)
```

Multiplying an integer and a `WizCoin` object is commutative: `2 * purse` is the same as `purse * 2`. Instead of copying and pasting the code from `__mul__()`, we just call `self.__mul__()` and pass it the `other` parameter.

After updating `wizcoin.py`, practice using the reflected multiplication dunder method by entering the following into the interactive shell:

```
>>> import wizcoin
>>> purse = wizcoin.WizCoin(2, 5, 10)
>>> purse * 10 # Calls __mul__() with 10 for the `other` parameter.
WizCoin(20, 50, 100)
>>> 10 * purse # Calls __rmul__() with 10 for the `other` parameter.
WizCoin(20, 50, 100)
```

Keep in mind that in the expression `10 * purse`, Python first calls the `int` class's `__mul__()` method to see whether integers can be multiplied with `WizCoin` objects. Of course, Python's built-in `int` class doesn't know anything about the classes we create, so it returns `NotImplemented`. This signals to Python to next call `WizCoin` class's `__rmul__()`, and if it exists, to handle this